

诚信声明

我声明，所呈交的毕业论文是本人在老师指导下进行的研究工作及取得的研究成果。据我查证，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得其他教育机构的学位或证书而使用过的材料。我承诺，论文中的所有内容均真实、可信。

毕业论文作者签名：

签名日期： 年 月 日

基于区块链完整性保证的电子供应链系统

摘要：随着电子产品供应链不断向全球化与复杂化演进，传统管理方式在数据共享、信息可信性以及产品追溯能力方面逐步显现出不足，存在信息孤岛现象突出、数据易被篡改以及质量问题难以精确定位等。这些问题不仅提升了供应链协同成本，也对产品质量监管与安全保障提出了更高要求。针对上述挑战，对区块链技术进行了研究，设计并实现了一种面向电子产品全生命周期的供应链追溯系统。系统以联盟链平台为依托，结合智能合约以及链上存证、链下存储的混合架构，实现了覆盖设计、生产、组装、流通、销售直至报废阶段的全流程数据可信记录与追溯管理。在模型构建方面，提出基于物理唯一标识（ECID）与产品序列号（SN）的双标识机制，建立部件、整机、流通的多层级映射关系，从而同时支持正向跟踪与反向追溯。实验与应用结果表明，该系统能够有效提升供应链数据透明度与追溯能力，降低假冒伪劣产品流通风险，并为质量问题的精准定位及召回提供有力支撑，同时为区块链技术在工业供应链领域的应用提供了实践参考。

关键词：区块链；供应链追溯；电子产品；智能合约

Electronic Supply Chain System Based on Blockchain Integrity Assurance

Abstract: As electronic product supply chains continue to evolve toward greater globalization and complexity, traditional management approaches have gradually revealed shortcomings in data sharing, information credibility, and product traceability. Prominent issues include the existence of information silos, vulnerability to data tampering, and difficulties in accurately locating quality problems. These challenges not only increase the cost of supply chain collaboration but also impose higher requirements on product quality supervision and safety assurance. To address these issues, this study investigates blockchain technology and designs and implements a supply chain traceability system covering the entire lifecycle of electronic products. The system is built on a consortium blockchain platform and adopts a hybrid architecture combining smart contracts, on-chain evidence storage, and off-chain data storage. It enables trustworthy data recording and traceability management throughout the full process, including design, manufacturing, assembly, circulation, sales, and end-of-life stages. In terms of model construction, a dual-identifier mechanism based on a physical unique identifier (ECID) and product serial number (SN) is proposed. A multi-level mapping relationship among components, complete products, and circulation processes is established, thereby supporting both forward tracking and backward traceability. Experimental and application results demonstrate that the proposed system effectively enhances supply chain data transparency and traceability, reduces the risk of counterfeit and substandard products entering circulation, and provides strong support for precise quality issue localization and product recalls. Furthermore, it offers practical insights for the application of blockchain technology in industrial supply chains.

Keywords: Blockchain; Supply Chain Traceability; Electronic Products; Smart Contracts

目 录

1 绪论	6
1.1 研究背景和意义	6
1.2 研究现状	7
1.2.1 电子产品追溯研究	7
1.2.2 区块链追溯研究	8
1.3 研究内容与工作	10
1.4 论文组织结构	11
2 电子产品供应链与区块链技术	12
2.1 电子产品供应链	12
2.1.1 供应链概述	12
2.1.2 供应链面临的挑战	13
2.2 区块链概述	14
2.2.1 区块链工作原理	14
2.2.2 联盟链	15
2.2.3 共识机制	15
2.2.4 智能合约	17
2.2.5 签名机制	17
2.3 区块链平台选型	18
2.3.1 FISCO BCOS 整体架构	18
2.3.2 群组与权限模型	19
2.3.3 交易流程	20
2.4 去中心化存储技术	21
3 全流程可追溯供应链系统功能设计	23
3.1 系统需求分析	23
3.1.1 功能性需求	23
3.1.2 非功能性需求	25
3.2 系统角色与权限设计	27
3.3 系统总体架构设计	28
3.4 核心业务流程设计	29
3.4.1 订单与协议流转设计	29
3.4.2 生产测试与 ECID 注册流程设计	30
3.4.3 部件到整机组装映射设计	31
3.4.4 分销流通与物流状态变更设计	32
3.4.5 售后与生命周期闭环设计	33

4 全流程可追溯供应链系统实现	35
4.1 开发环境部署与技术栈选型	35
4.1.1 区块链底层网络搭建	35
4.1.2 后端应用开发技术栈	36
4.2 核心智能合约的编写与部署	37
4.2.1 角色权限与资质审核合约实现	37
4.2.2 订单流转与生命周期管理合约实现	37
4.2.3 溯源与召回机制合约逻辑实现	39
4.3 用户前端与核心界面开发实现	40
4.3.1 注册模块	40
4.3.2 供应商模块	41
4.3.3 制造商模块	42
4.3.4 组装商模块	43
4.3.5 分销商模块	44
4.3.6 终端用户模块	45
4.3.7 监管机构模块	46
5 系统测试与分析	47
5.1 核心功能测试	47
5.2 性能测试	52
5.2.1 测试环境	52
5.2.2 测试指标	53
5.2.3 测试结果	54
6 总结与展望	55
6.1 工作总结	55
6.2 工作展望	56
致谢	58
参考文献	59

1 绪论

1.1 研究背景和意义

现代电子产品供应链是一个高度复杂、全球化且多层级交织的网络。一个完整的电子产品的诞生，往往需要经历集成电路设计、印刷电路板制造、元器件组装，再到整机分销与最终使用的漫长链路。在这个过程中，涉及供应商、制造商、组装商、分销商及监管机构等众多独立参与方。

2016 年，三星 Galaxy Note 7 手机曾因电池缺陷引发多起起火事故，由于供应链追溯系统响应迟缓，未能及时从海量供应商中锁定引发热失控的具体电池隔膜批次，导致了全球范围内的盲目大召回，不仅造成了数百亿美元的直接经济损失，更严重反噬了品牌信誉。在复杂的组装层级下，一旦终端产品出现质量缺陷，核心企业极难在短时间内精准定位问题元器件的批次与来源^[1]。

此外，传统电子产品供应链管理模式的正面临“信息孤岛”和“信任不足”的双重挑战。各参与主体往往将关键数据保存在各自的中心化系统中，出于商业保密及数据安全等因素考虑，彼此之间缺乏有效的数据共享机制^[2]。在 2020—2022 年全球半导体供应链“缺芯”危机期间，大量翻新、未经授权以及未通过安全合规检测的劣质芯片流入市场。由于传统纸质或电子资质证明易被篡改，且质检机构的抽检结果难以及时同步并实现有效防伪，导致系统级设备存在较大的安全风险隐患。

与此同时，传统供应链在产品生命周期管理方面也存在明显不足。现有管理模式通常停留在销售阶段，对产品后续的使用情况、保修过程以及最

终报废与处置缺乏持续跟踪。这不仅容易引发售后纠纷，也难以满足日益严格的电子废弃物环保监管要求。

区块链作为一种新兴的分布式账本技术，依托其去中心化、不可篡改、可追溯以及智能合约自动执行等特征，为上述问题的解决提供了新的技术路径。具体而言，区块链通过共识机制实现多方共同参与的数据维护与共享，借助密码学手段保障数据的安全性，并利用链式结构使数据一旦写入便难以被篡改^[3]。此外，智能合约能够按照预设规则自动执行相关业务逻辑，从而在一定程度上提升供应链管理的自动化水平与可信性。

在此背景下，构建一种基于区块链的电子产品全流程追溯系统，具有较为突出的理论意义与现实价值。从理论层面来看，将区块链技术引入电子产品供应链管理，有助于拓展其在工业领域的应用边界，为可信计算及分布式系统研究提供新的实践场景^[4]。从实际应用角度来看，通过搭建涵盖供应商、制造商、组装商、分销商及监管机构等多方主体的协同体系，可以提升供应链整体透明度，实现产品全生命周期的可追溯管理，进而降低假冒伪劣产品流通风险，增强产品质量监管能力。

1.2 研究现状

随着全球电子产品市场规模的增大，近年围绕电子产品追溯机制以及区块链技术的应用已有大量的研究积累，为本系统的设计与实现提供了理论参考与技术参照。

1.2.1 电子产品追溯研究

在早期阶段，电子产品的追溯主要依赖条形码和射频识别（Radio

Frequency Identification, RFID) 等技术手段。通过为产品或关键元器件设置唯一标识, 并在生产、仓储以及物流环节进行扫描记录, 从而实现对基本流转信息的采集与管理^[5]。例如, 在半导体制造过程中, 通常利用批次号或序列号对芯片进行标识, 以便在一定范围内追踪其来源及生产流程。

随着信息技术的不断发展, 物联网 (Internet of Things, IoT) 逐步被引入到电子产品追溯系统中。借助各类传感器及嵌入式设备, 可以对生产环境、设备运行状态以及物流过程中的信息进行实时采集, 从而实现更为细致的数据记录。例如, 在电子制造环节中, 通过环境传感器对温湿度、电压波动等关键参数进行监测, 并将相关数据与产品标识进行关联, 有助于提升质量问题分析的准确性。同时, 将 RFID 技术与 IoT 相结合的追溯方案, 还能够实现数据的自动化采集, 减少人工干预带来的误差, 提高整体数据的可靠性^[6]。

1.2.2 区块链追溯研究

随着传统供应链追溯系统在数据可信性以及跨主体协同方面逐步暴露出不足, 区块链技术开始被视为提升追溯可靠性的一个重要研究方向。

在相关研究的早期阶段, 学者多将区块链应用于食品安全和物流追溯场景。例如, 通过将区块链与物联网 (IoT) 技术相结合, 利用传感器对产品状态进行实时采集, 并将关键数据记录至链上, 从而实现从生产到消费全过程的可视化与透明化管理。这类研究普遍认为, 区块链在一定程度上能够缓解传统追溯系统中数据造假以及责任界定不清等问题^[8]。此外, 也有研究提出将区块链与 RFID 技术融合的农产品追溯方案, 通过把 RFID 标签所

采集的数据写入区块链，实现信息的防篡改与可验证共享。

随着研究不断深入，区块链在追溯系统中的应用逐渐由单一的数据存证扩展到多方协同以及业务流程自动化层面。其中，智能合约被广泛引入，用于支持数据自动记录、访问权限控制以及事件触发等功能。例如，在供应链场景下，订单生成、生产记录、物流流转以及销售数据等环节，均可以通过智能合约完成自动执行并同步上链，从而减少人为干预，提高整体运行效率^[9]。同时，为应对区块链在存储能力和扩展性方面的限制，一些研究提出“链上存证+链下存储”的混合模式，即将大规模数据存储于分布式文件系统中，仅将其摘要信息写入区块链，以降低存储成本并提升系统扩展能力^[10]。

在工业及电子产品领域，区块链追溯研究逐步转向复杂供应链环境中的可信协同问题。由于电子产品供应链涉及主体众多、流程复杂且生命周期较长，传统系统往往难以实现跨组织的数据共享与统一管理。相关研究指出，引入联盟链架构可以在保障数据隐私的前提下，实现多节点之间的协同记账，不仅提升了系统性能，也增强了数据的可信程度^[11]。

尽管区块链在追溯应用中展现出一定优势，但现有研究仍存在一些不足。例如，其性能和吞吐能力在高频业务场景下仍显不足，不同区块链系统之间的互操作性有待提升，同时链上数据的隐私保护机制仍需进一步完善。因此，在兼顾系统性能的前提下实现数据安全共享与隐私保护，依然是区块链追溯领域中待解决的关键问题。

1.3 研究内容与工作

针对电子产品供应链中普遍存在的“信息孤岛”、“数据易被篡改”以及“追溯困难”等问题，本文以区块链技术为核心基础，结合智能合约与离链存储机制，设计并实现了一套覆盖全流程的电子产品供应链追溯管理系统。本文的主要研究内容及完成工作如下：

（1）在分析电子产品完整生命周期的基础上，对供应商、制造商、组装商、分销商以及监管机构等多方主体之间的数据交互关系进行了系统梳理，并总结其在信任机制方面的关键问题，从而明确系统在数据可信性与协同共享方面的核心需求。

（2）提出一种结合物理唯一标识（ECID）与产品序列号（SN）的双标识方案，构建“部件—整机—流通”之间的多层级映射关系，实现由单一元器件到整机产品的全链路追踪。同时，基于链上数据设计反向追溯机制，使系统在出现质量问题时能够快速定位问题批次并支持精准召回。

（3）围绕供应链中的关键业务流程，完成了智能合约的设计与实现，对生产订单（`ProductionRequest`）、设备记录（`DeviceRecord`）、组装记录（`AssemblyRecord`）、销售记录（`SaleRecord`）以及报废记录（`DecommissionRecord`）等核心数据结构进行建模，从而支持订单流转、生产登记、组装映射、物流跟踪以及召回管理等功能的自动化执行。

（4）提出“链上存证+链下存储”的混合架构方案，在联盟链环境下将关键业务摘要及操作凭证记录于链上，同时将完整业务数据存储于链下数据库及分布式存储系统中，并基于 `Spring Boot` 框架实现了配套的前后端管

理系统。

1.4 论文组织结构

第一章，绪论。本章主要介绍研究的背景与意义，分析电子产品供应链中存在的典型问题，并对国内外相关研究现状进行梳理，在此基础上明确本文的研究内容以及整体结构安排。

第二章，电子产品供应链与区块链技术。本章首先对电子产品供应链的基本结构进行分析，随后对区块链相关核心技术进行系统阐述，包括其基本原理、联盟链架构、共识机制、智能合约以及数字签名等内容。

第三章，全流程可追溯供应链系统功能设计。在完成需求分析的基础上，本章对系统的整体架构及功能模块进行设计，明确各类角色及其权限划分，详细描述订单管理、生产登记、组装关系映射、物流跟踪以及溯源查询等关键业务流程，同时给出智能合约的数据结构设计方案。

第四章，全流程可追溯供应链系统实现。本章重点说明系统的实现过程，涵盖区块链网络部署、智能合约开发、后端服务构建以及前端界面设计等内容，并对关键功能模块的实现方法及相关技术细节进行介绍。

第五章，系统测试与分析。本章从功能与性能两个方面对系统进行测试，分析其在不同业务场景下的运行表现，对系统的稳定性与扩展能力进行评估，并对测试结果进行总结。

第六章，总结与展望。本章对全文的研究工作与主要创新点进行归纳，分析当前系统仍存在的不足之处，并对后续可能的改进方向及进一步研究内容进行展望。

2 电子产品供应链与区块链技术

2.1 电子产品供应链

2.1.1 供应链概述

电子产品从设计到终端产品的完整流程通常包括以下几个关键阶段^[12]。首先是 IP 设计与验证阶段，供应商完成集成电路或印刷电路板设计图纸。其次是晶圆制造阶段。制造商在接收到设计文件后，通过光刻、刻蚀、离子注入、金属化等数百道工艺步骤在硅晶圆上制造集成电路。制造过程中需要记录每一片晶圆的生产批次号、工艺参数和良率数据。第三是封装与测试阶段。晶圆制造完成后，芯片被运送至封装测试厂，进行划片、键合、塑封、打标等工序，并在不同温度条件下进行功能和性能测试。第四是板级组装阶段。组装商将经过测试的芯片和其他元器件焊接到印刷电路板上，加载固件，进行整机功能测试，最终赋予产品唯一的序列号（SN）。组装完成后，产品进入分销渠道，经由多级分销商最终到达终端用户手中。当产品到达生命周期终点时，还需经由回收商进行报废或环保处置，形成从“摇篮到坟墓”的完整闭环。

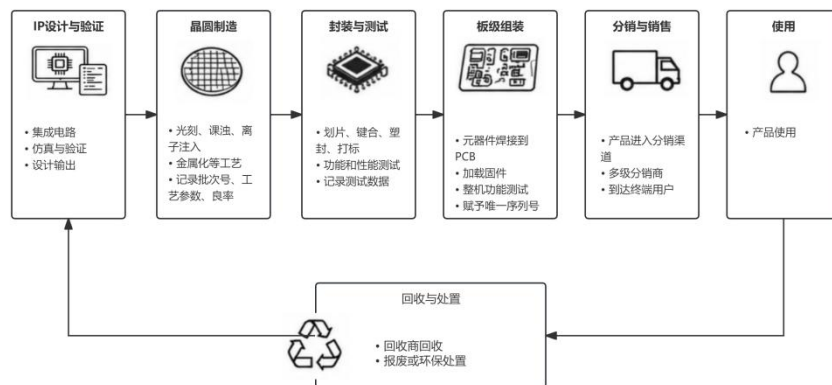


图 2-1 电子产品生命周期流程图

2.1.2 供应链面临的挑战

电子产品供应链在全球化分工背景下虽然显著提升了生产与流通效率，但从产业结构角度来看，也不可避免地引发了信任缺失与信息割裂等一系列问题。

（1）数据孤岛问题。在传统供应链管理体系中，各参与主体通常只围绕自身业务建立信息系统。供应商向制造商提供物料，制造商再将产品交付给组装商，随后由组装商流向分销商，各节点仅掌握有限的上下游信息，难以形成对产品全流程的整体认知。一旦出现质量问题并需要开展追溯工作，往往需要跨越多个企业系统进行人工沟通与数据整合，周期可能长达数周甚至数月^[12]。

（2）假冒伪劣组件问题。假冒集成电路长期存在于电子产品供应链中，并且治理难度较大。由于集成电路体积微小、外观高度相似，其真实性难以通过直观方式判断。不法主体通常通过回收废旧电路板中的芯片，对其进行翻新处理，例如重新打磨标识，并伪装成高规格或新批次元器件再次流入市场^[13]。

（3）灰色市场交易问题。除官方授权分销体系之外，灰色市场在一定程度上能够缓解库存积压问题，但同时也为假冒产品进入供应链提供了隐蔽渠道。尤其在芯片供应紧张的阶段，部分制造商不得不依赖非授权来源采购元器件，从而进一步加剧了潜在风险。

（4）责任追溯困难。当产品质量问题发生时，由于供应链各环节之间

数据彼此独立、记录标准不统一，加之纸质文件或电子表格易被篡改或遗失，责任主体的定位往往十分复杂。一些情况下，由于无法准确锁定问题来源，企业只能选择对相关批次产品进行全面召回，进而带来巨大的经济损失。更为严重的是，如果问题元器件的来源与影响范围无法及时确认，还可能对消费者安全形成持续性隐患。

2.2 区块链概述

区块链最早是由中本聪在 2008 年《比特币：一种点对点的电子现金系统》中提出^[3]，其本质是一种去中心化的分布式账本，通过密码学来保证数据的不可篡改和伪造。本节从区块链的工作原理、联盟链、共识机制、智能合约及签名机制五个方面阐述。

2.2.1 区块链工作原理

区块链由一系列区块连接而成，其按时间排序并以链式结构连接。每个区块包含区块头与区块体两部分。区块头封装了区块的版本号、前一区块的哈希值、时间戳、Merkle 树根哈希值以及工作量证明参数等信息；区块体包含交易数据列表。由于区块存储前一区块的哈希值，所以任何对历史数据的篡改都会破坏后续区块的哈希值，从而被其他节点检测到以维持其不可篡改性^[14]。

区块链网络采用点对点架构，各节点维护相同的账本副本，当一笔新交易被广播到网络中时，具有记账权的节点将该交易与其他待确认交易打包成一个新区块，经其他节点验证后附加到已有区块链上。

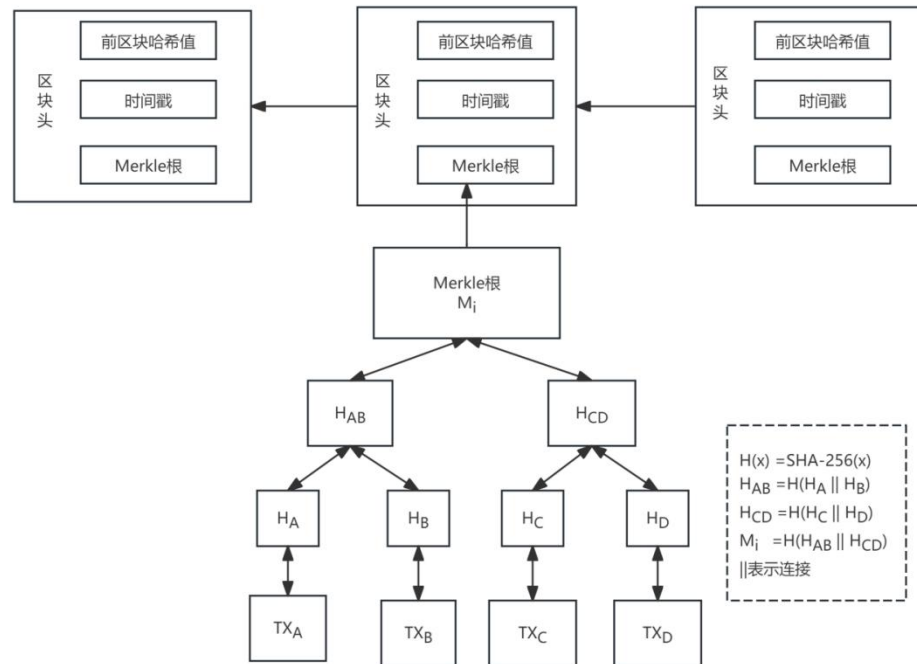


图 2-2 区块链架构

2.2.2 联盟链

区块链可分为公有链、联盟链和私有链。公有链对任何节点开放，完全去中心化；私有链由单一组织控制，只有内部节点有写入权限。

联盟链介于公有链与私有链之间，其在公有链的基础上引入节点准入机制，只有经过认证的节点才能接入网络。这使得联盟链的节点数量可控，显著提升了交易吞吐量，也更有利于系统的管理与维护。典型的认证手段是由证书颁发机构（Certificate Authority, CA）为每个参与节点签发数字证书，凭借证书完成准入验证^[11]。

2.2.3 共识机制

共识机制是区块链的核心技术之一，其使分布式网络中互不信任的节点达成数据状态一致。在区块链网络中，没有单一的中心化权威机构，所以

必须通过共识算法来保证所有节点认可账本状态^[15]。

主流的共识算法可根据是否具有拜占庭容错能力划分。拜占庭容错问题描述的是在分布式系统中存在恶意节点的情况下，诚实节点如何达成一致的问题。以下介绍几种典型共识算法：

（1）工作量证明（**Proof of Work, PoW**）：节点通过消耗大量算力求解一个数学难题来竞争记账权，最先解出难题的节点获得新区块打包权并获得奖励。**PoW** 虽然安全程度高但耗能巨大且共识效率低。

（2）权益证明（**Proof of Stake, PoS**）：在该机制下，节点获取记账权的概率通常与其所持代币数量及持有时长相关，不再依赖类似 **PoW** 的算力竞争方式。通过这种方式，在降低能源消耗的同时，也在一定程度上提升了共识过程的执行效率。

（3）实用拜占庭容错（**Practical Byzantine Fault Tolerance, PBFT**）：**PBFT** 采用预准备、准备与提交三个阶段完成消息在节点间的传播与校验，可在系统中最多存在不超过三分之一恶意节点的情况下仍保持正常运行，因此在联盟链环境中得到广泛应用。其通信复杂度为 $O(n^2)$ ，在节点数量较少时能够表现出较高性能，但随着规模扩大，通信开销迅速增加，从而对系统扩展性形成一定制约^[16]。

（4）**Raft** 共识算法：**Raft** 属于一种不具备拜占庭容错能力的一致性算法，主要依赖 **Leader** 选举、日志复制以及一系列安全约束来保证系统状态一致。相较于 **PBFT**，其通信复杂度为 $O(n)$ ，具备更低的共识延迟和更高的吞吐能力，但无法应对恶意节点行为。因此，**Raft** 更适用于节点之间具

备较高信任基础的联盟链场景,或与 PBFT 等算法结合使用以增强系统鲁棒性。

综合当前联盟链的实际应用情况,具体共识机制的选取往往取决于场景需求:当系统对拜占庭容错能力要求较高时,通常优先考虑 PBFT 或其改进方案;而在参与节点可信度较高的前提下,Raft 凭借其性能优势往往成为更合适的选择。

2.2.4 智能合约

智能合约最早由 Nick Szabo 于 1995 年提出。在区块链语境下,智能合约部署在区块链上,当满足预设条件时,能够自动执行的程序代码。

以太坊是目前应用最广泛的智能合约区块链平台,以太坊的智能合约主要由 Solidity 语言编写,在以太坊虚拟机(Ethereum Virtual Machine, EVM)上运行。当外部账户向合约账户发起交易时,EVM 根据交易输入触发合约函数,所有节点独立运行该合约代码并对执行结果达成共识。

对于本课题中的电子产品供应链系统,智能合约承担着核心业务逻辑的实现。供应商发布 ProductionRequest、制造商签署 ManufacturingAgreement 并在链上注册 DeviceRecord、组装商创建 AssemblyRecord、分销商记录 TransferEvent、监管机构触发召回通告等,均可通过智能合约实现业务流程的自动化与链上事件存证。

2.2.5 签名机制

在比特币和以太坊等区块链平台中,交易所采用的主要数字签名算法是椭圆曲线数字签名算法(Elliptic Curve Digital Signature Algorithm, ECDSA)。

相较于传统 RSA 算法，ECDSA 有更短的密钥长度和更快的执行速度。

发送方生成一个随机数，与私钥结合对交易信息的哈希值计算，生成签名 (r,s) ；接收方使用发送方的公钥和签名值对交易进行验证。基于椭圆曲线离散对数问题，在不知道私钥的情况下伪造有效签名在计算上是不可行的^[17]。

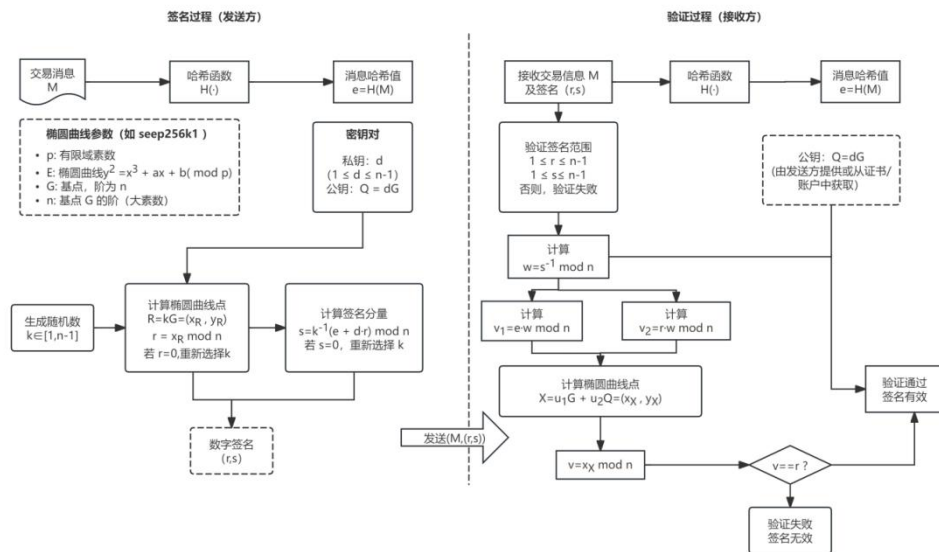


图 2-3 ECDSA 签名流程

2.3 区块链平台选型

在诸多区块链平台中，本系统选择 FISCO BCOS 作为底层区块链基础设施。本节从整体架构、群组与权限模型、交易流程三个维度阐述其技术特性与选型理由。

2.3.1 FISCO BCOS 整体架构

FISCO BCOS 是由微众银行牵头金融区块链合作联盟（FISCO）开源的企业级联盟链底层平台，兼具高性能、高安全性和易扩展等特性。整体架构划分为基础层、核心层、管理层、接口层^[18]。

- (1) 基础层：提供区块链的基础数据结构和算法库。
- (2) 核心层：核心层分为两个部分，分别为链核心层与互联网核心层。链核心层负责链式数据结构的维护、交易执行引擎调度与存储驱动管理；互联网核心层负责承载 P2P 网络通信、共识机制和区块同步机制。
- (3) 管理层：管理层实现区块链的参数配置、账本管理和 AMOP（Advanced Messages Onchain Protocol, AMOP）。
- (4) 接口层：面向区块链用户，提供支持多种协议的 RPC 接口、多语言 SDK 以及交互式控制台^[18]。

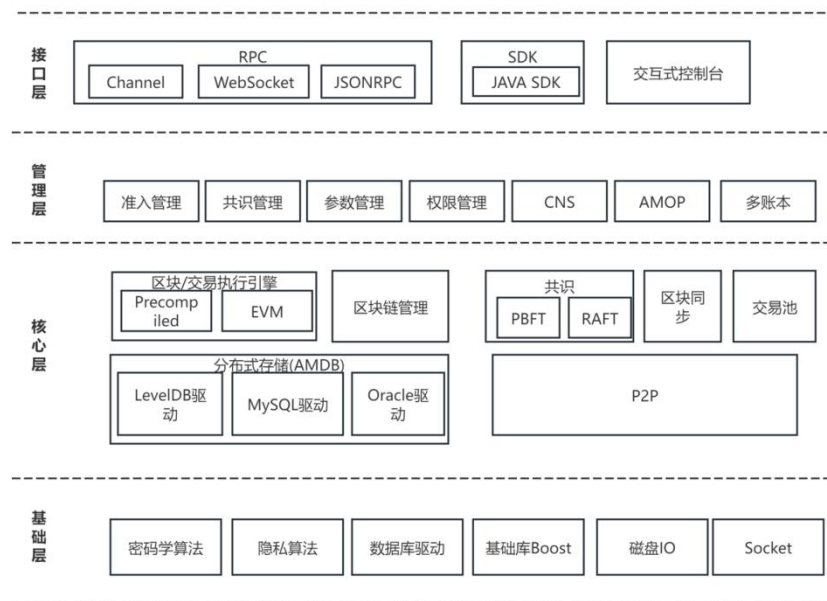


图 2-4 FISCO BCOS 整体架构

2.3.2 群组与权限模型

FISCO BCOS 从 2.0 版本开始引入多群组架构，允许部分节点独立形成一个账本，组内进行节点的共识和存储操作。逻辑上，每个群组相当于一条独立链。单个节点可以同时加入多个群组，参与多条链的共识与记账。

群组架构具备交易隔离、并行执行、共识独立、网络隔离等关键属性。目前 FISCO BCOS 主要支持两种共识算法：PBFT 和，若参与方之间信任基础较强，可以选用 Raft 来获得更高吞吐性能；若信任基础较弱，可以选用 PBFT 来确保系统的拜占庭容错能力。

FISCO BCOS 提出了权限控制模型（Account Role Permission Interface, ARPI），即账号、角色、权限、接口。ARPI 模型中，账号和角色为多对一关系；角色与权限为多对多关系，一个角色可拥有多个权限集合，一个权限也能被多个角色拥有。在实际操作中，同一角色下可能有多个账号，每个账号使用独立的公私钥对，交易时使用私钥进行签名，接收方通过公钥验证发出方身份^[18]。

2.3.3 交易流程

FISCO BCOS 交易流程分为五个阶段：交易提交、交易打包、区块验证与共识、区块上联、交易确认^[18]。

（1）交易提交：用户通过 SDK 或 curl 命令向区块链节点发起 RPC 请求，节点收到交易后，检查交易池是否已满。若交易池未满，则将交易附加到交易池中，最后将交易广播。

（2）交易打包：打包器（Sealer）从交易池中取出交易，当交易数量达到阈值或超时时间后，将收到的交易封装为一个新区块到共识引擎。

（3）区块验证与共识：共识引擎收到新区块后，调用区块验证器（BlockVerifier）对区块校验。如果区块合法则进一步调用执行引擎（Executor）执行区块中的交易。同时通过共识协议与各节点达成一致，并

将区块发送至区块链管理模块（BlockChain）。

（4）区块上链：BlockChain 对区块信息进行最终检查，确认无误后将区块数据和执行上下文中的表数据整合为一个原子事务，提交到底层存储中，完成区块上链。

（5）交易确认：区块上链后，客户端通过查询节点获取交易状态和结果。

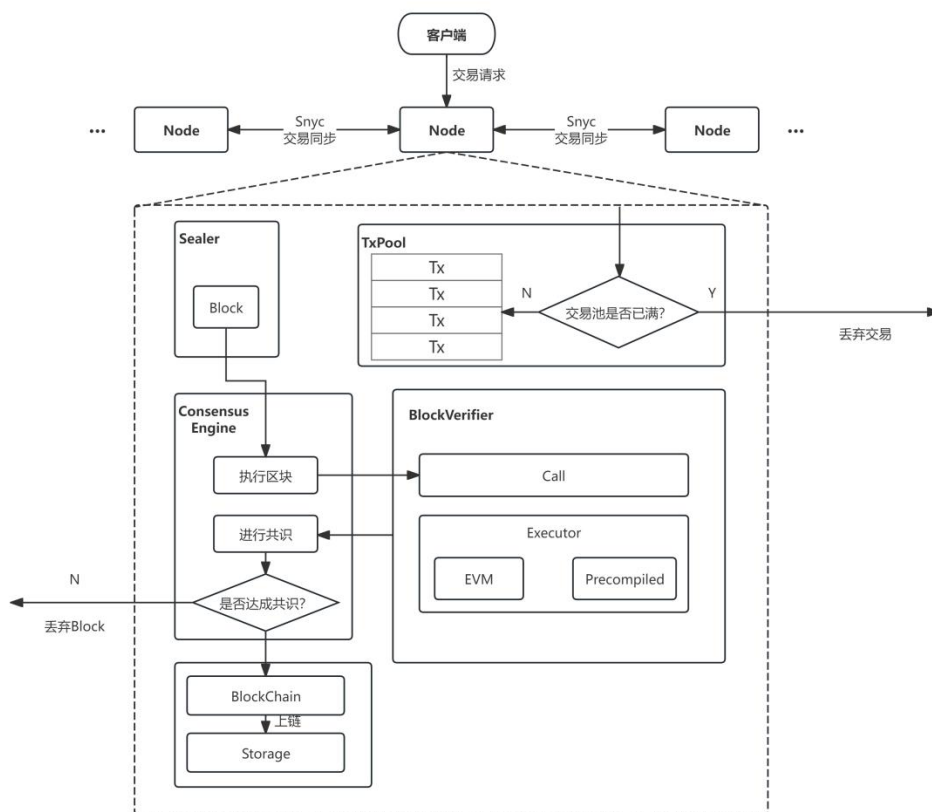


图 2-5 FISCO BCOS 交易流程

2.4 去中心化存储技术

区块链在链上存储方面存在容量受限且成本较高的问题。以 FISCO BCOS 为例，单笔交易可承载的数据通常仅为几十 KB，而在供应链场景中，诸如设计图纸等文件往往达到数 MB 甚至数百 MB，二者在规模上存在明显

差距。

星际文件系统（InterPlanetary File System, IPFS）是一种基于点对点网络的分布式存储协议，其核心特点是以内容寻址替代传统 HTTP 中的位置寻址机制。在传统 Web 体系中，用户依赖 URL（统一资源定位符）访问资源，一旦文件位置发生变化或服务器不可用，对应链接即失效，资源也难以获取。相比之下，IPFS 通过内容标识符（Content Identifier, CID）定位数据，该标识符由文件内容的密码学哈希计算生成，具有唯一性。当用户请求某一 CID 时，网络中任意持有该数据副本的节点均可返回结果，这种机制使系统具备分布式冗余与断点续传能力，从而在数据可用性与抗审查性方面表现更优^[19]。

在文件写入 IPFS 节点时，系统通常先将原始文件拆分为多个数据块，并分别计算其多重哈希值，随后将这些数据块按照 Merkle DAG 结构进行组织，最终生成一个根 CID 作为文件的唯一标识。由于该标识直接依赖于内容本身，即便是极小的修改，也会导致对应 CID 发生变化。

3 全流程可追溯供应链系统功能设计

3.1 系统需求分析

3.1.1 功能性需求

本系统旨在构建一个覆盖电子产品从设计、生产、检测、销售、使用直至报废全生命周期的可追溯管理平台。基于区块链与智能合约技术，系统需满足以下核心功能性需求：

（1）用户注册与资质审核

系统应能够支持供应商、制造商、组装商、分销商、监管机构、质检机构以及终端用户等多类型角色的注册与登录。针对供应商的接入，需要设置较为严格的资质审核机制：供应商提交营业执照、相关资质证明等材料后，由监管机构在本地完成审核，并同步更新资质信息库；与此同时，将审核结果的摘要信息（包括文件哈希值及审批状态）写入区块链，从而保证审核过程具备不可篡改性及可追溯性。

（2）订单全生命周期管理

供应商可以发起生产订单（**ProductionRequest**），既可指定具体制造商，也可以选择面向全网发布。订单内容通常包括物料清单（**BOM**）哈希、设计文档哈希、生产数量、预期交付时间以及质量要求等信息。制造商在确认后签署制造协议（**ManufacturingAgreement**），协议原文存放于 IPFS 等链下存储系统，仅将其哈希值及双方签名写入区块链，从而实现订单从创建、签署到状态演变全过程的链上记录与留存。

（3）生产与 ECID 注册管理

制造商需要为每一个生产单元分配唯一的物理标识（ECID），并通过烧录或附加物理标签的方式与具体产品建立对应关系。生产完成后，制造商将 ECID、订单编号、批次信息、设备类型以及测试报告哈希等关键数据登记到区块链上，从而形成不可篡改的设备数字身份。对于检测不合格的产品，系统同样支持将不合格原因及处理方式（如退货或销毁）记录上链，并自动触发后续处置流程。

（4）部件到整机组装映射

组装商需记录部件来源（ECID 列表）、组装批次号、整机序列号（SN）、固件版本及整机测试报告哈希。智能合约建立“部件 ECID→整机 SN”的映射关系，支持后续溯源查询时快速定位部件来源与整机去向，为质量召回提供精准定位能力。

（5）分销流通与物流追踪

分销商在货物流转过程中上链记录物流事件（TransferEvent），包含物流单号、时间戳、发送方与接收方身份、流转类型（发货/收货/退货）等信息。系统支持单品级或批次级的流转登记，并更新当前货权归属，确保流通环节透明可查。

（6）销售与客户信息管理

销售环节需支持销售记录上链或摘要上链。当涉及终端消费者个人信息时，系统采用“链下存储加密数据、链上仅存客户身份哈希”的隐私保护策略，既满足追溯需求，又符合个人信息保护要求。

（7）溯源查询与验证

终端用户或监管机构可通过扫描二维码或输入 ECID/SN 查询产品全生命周期轨迹。前端页面需以时间线形式可视化展示“部件生产→组装→物流→销售”的完整路径，并支持点击拉取离链文件（设计图纸、检测报告等），同时自动校验文件哈希与链上记录的一致性，确保展示信息真实可信。

（8）质量召回与报废管理

当出现质量问题时，系统支持终端用户提交召回请求，监管机构或制造商可根据故障 ECID/SN 沿溯源链条反向定位受影响批次，触发召回通告。产品报废时，报废事件（Decommission）上链记录处置方式、时间及责任机构，实现产品生命周期的完整闭环。

3.1.2 非功能性需求

（1）数据完整性与防篡改

系统会对设计文档、BOM 清单、制造协议、质检报告以及物流事件、销售凭证、召回与报废记录等关键业务数据统一计算 SHA-256 摘要，并借助智能合约将其锚定至 FISCO BCOS 区块链。链上仅记录业务类型、摘要哈希、操作主体地址及时间戳等精简信息，完整业务数据则保存在链下数据库中，文件原文存储于 IPFS。通过“链上存证 + 链下数据”的组合方式，在后续查询时可重新计算文件哈希并与链上或数据库中的摘要进行比对，从而判断数据是否发生篡改。

（2）身份认证与访问控制

系统基于 Spring Security 与 JWT 机制实现用户身份认证，用户登录成

功后由前端保存令牌，并在后续请求中携带以完成身份校验。在权限设计方面，系统参考了经典的基于角色的访问控制（RBAC）模型，并结合区块链身份认证机制实现细粒度权限管理^[20]，根据不同用户角色动态提供相应的菜单与功能入口，涵盖供应商、制造商、组装商、分销商、监管机构及终端用户等类型。后端在处理关键业务流程时，会结合当前用户身份、数据归属关系以及供应商资质审核结果进行综合校验。例如，仅当供应商通过资质审核后，才允许其发布设计文档、BOM 及生产订单；而在物流与销售环节，则要求操作用户必须为产品当前的货权持有方。

（3）可扩展性与性能

鉴于区块链在处理大体量文件及复杂业务明细方面存在局限，系统采用链上与链下协同的数据存储模式。其中，MySQL 主要负责存储用户信息、订单、批次、ECID、组装记录以及物流、销售和召回等结构化数据；IPFS 用于承载设计图纸、协议文本、检测报告、资质材料及销售发票等大规模文件；而区块链则侧重记录关键数据摘要及交易凭证信息。

（4）隐私保护

系统遵循“敏感信息不直接上链、链上仅记录摘要”的设计思路。在处理企业资料、协议文本、质检报告、销售发票及消费者信息时，仅将文件哈希、客户标识哈希或业务摘要写入区块链，原始数据则存储于链下数据库或 IPFS 中。在销售环节，系统支持匿名与实名两种模式：匿名情况下不记录客户姓名与联系方式，仅生成对应的摘要哈希；在非匿名模式下，客户姓名和电话以编码形式保存在本地，并通过客户哈希实现后续的关联

与校验。

(5) 可用性与易用性

系统基于 Vue 与 Element Plus 构建 Web 操作界面，不同角色登录后按权限展示相应业务模块，从而减少跨角色操作带来的干扰。终端用户可通过公开的溯源页面输入产品 SN，查询装配信息、部件 ECID、物流与销售记录以及文件校验结果；监管机构则可访问资质审核、抽检管理、召回处理及审计取证等功能模块。此外，系统在对外溯源接口中设置了访问频率控制机制，以防止接口被恶意高频调用。

3.2 系统角色与权限设计

本系统涉及六类核心角色，根据供应链实际运作模式划分职能与权限，如表 3-1 所示。

表 3-1 系统角色职能

角色	标识	核心职能
供应商	Supplier	提供电路板设计，发布 BOM 与生产订单
制造商	Manufacturer	接收订单并生产电路板，赋予 ECID 并进行测试
组装商	Assembler	将部件组装为整机，赋予 SN，进行整机测试
分销商	Distributor	负责各环节间货物流通与销售
监管机构	Regulator	审核供应商资质，执行抽检，发布召回通告
终端用户	EndUser	扫码验证产品真伪与溯源，提交投诉召回请求

智能合约中通过 `roleOf` 映射维护各区块链地址对应的角色编码（1~6），函数调用时通过 `onlyRole` 修饰器进行权限校验，确保业务流程只能由合法角色驱动。系统前端与后端同样基于角色实施菜单与 API 接口的访问控制。

3.3 系统总体架构设计

系统采用分层架构设计，自下而上分为存储层、区块链层、后端服务层与应用层，如图 3-1 所示。

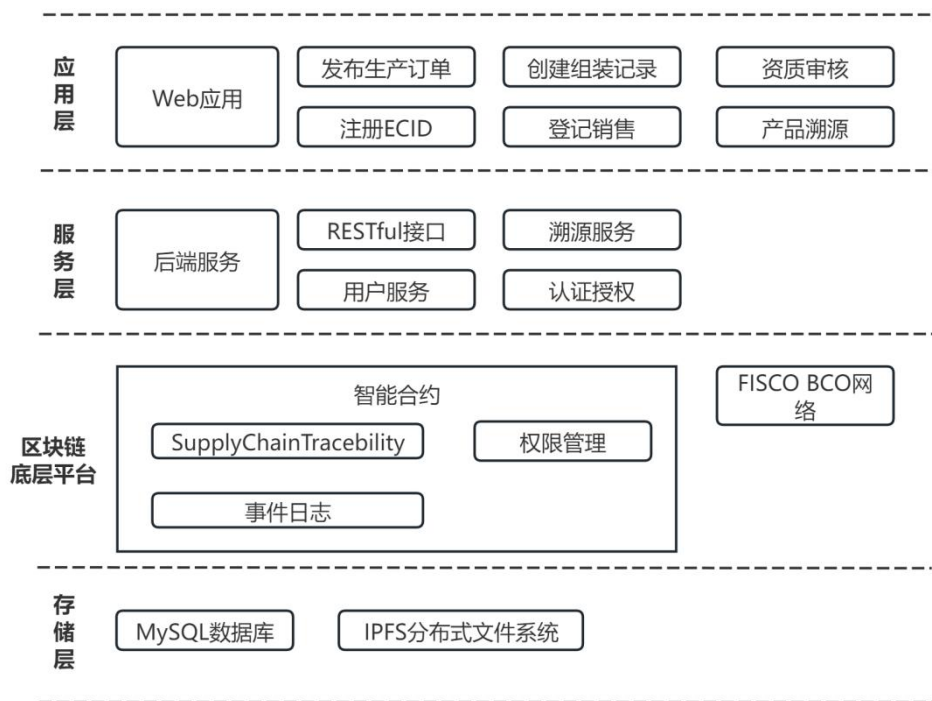


图 3-1 系统总体架构图

（1）应用层：采用 Vue.js 与 Element Plus 构建基于浏览器的 Web 管理界面，为供应商、制造商、组装商、分销商、监管机构及终端用户提供差异化的功能面板。前端主要负责表单录入、文件上传、业务数据展示和溯源结果可视化，并通过 RESTful API 与后端服务交互。

（2）后端 API 层：基于 Spring Boot 构建 RESTful API 服务，负责登录认证、权限校验、业务流程编排、数据库读写、文件哈希计算、IPFS 文

件上传下载以及区块链交易调用等功能。后端通过 JWT 维护登录状态，并根据用户角色控制业务接口访问。

（3）区块链服务层：系统选用 FISCO BCOS 作为底层联盟链平台，部署 SupplyChainTraceability 智能合约。后端通过 FISCO BCOS Java SDK 连接区块链节点，调用合约中 anchor、setRole、createProductionRequest、signManufacturingAgreement、registerDeviceRecord、createAssemblyRecord 等方法，实现关键业务摘要、角色映射和核心流转事件的链上存证。区块链网络由供应链参与方和监管机构共同维护，形成多方协同的可信存证环境。

（4）存储层：MySQL 关系数据库用于保存用户、订单、BOM、生产批次、ECID、组装记录、物流、销售、召回和操作日志等结构化业务数据，并通过 tx_hash 字段与链上交易记录关联。IPFS 用于保存设计图纸、资质材料、协议文件、检测报告、发票等大文件，系统保存其 CID 与 SHA-256 文件哈希，用于后续完整性校验。

3.4 核心业务流程设计

3.4.1 订单与协议流转设计

供应商在通过资质审核后，可依据设计文档与 BOM 物料清单发起生产订单。订单生成时，系统会分配全局唯一的订单编号，并将其初始状态设定为“待接单”。设计文档、BOM 文件及质量要求等内容经过哈希处理后，与订单编号、生产数量、预期交付时间等关键信息一并提交至智能合约，形成订单创建的链上交易记录。同时，交易哈希会回写至订单表，以支持

后续的审计与溯源。

在制造商接单环节，需要上传已签署的制造协议文件，并填写最终报价及承诺交付日期。系统会对订单是否存在、当前状态是否为“待接单”，以及制造商是否具备相应权限进行校验。协议文件经存证服务生成哈希值及存储地址后，调用智能合约完成签署记录的上链。接单完成后，订单状态由“待接单”更新为“已接单”，并同步写入链上状态信息，其交互过程如图 3-2 所示。

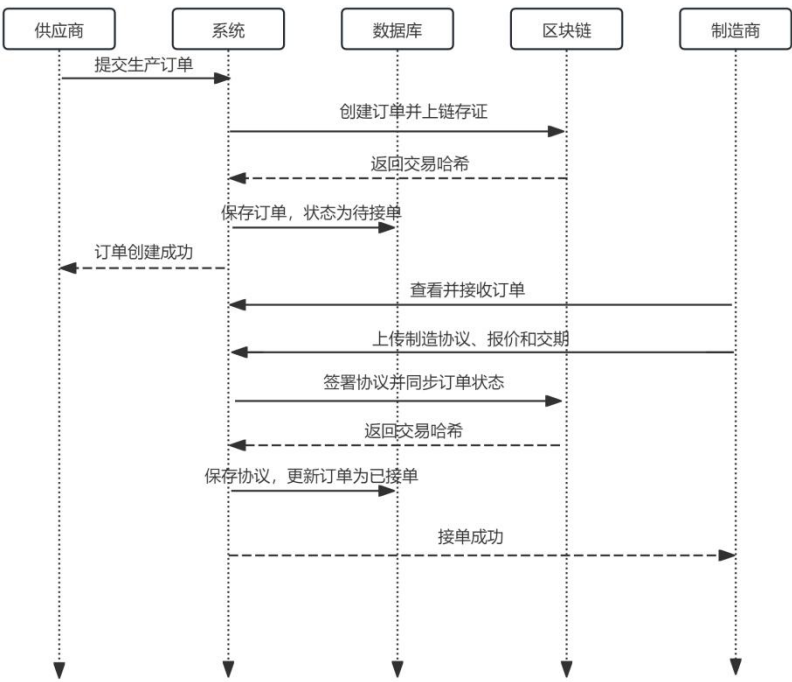


图 3-2 订单与协议流转时序图

3.4.2 生产测试与 ECID 注册流程设计

制造商在完成接单后，会依据订单信息建立对应的生产批次，并将批次与订单编号、制造商编号、计划产量以及 BOM 明细进行关联。每个 ECID 对应唯一的设备记录，其初始状态设定为“已生产”，同时记录订单编号、批次编号、制造商信息、设备类型及生产时间等数据。生产结束后，制造

商需上传测试报告并登记质检结果，只有检测合格的部件才允许进入后续链上登记流程。校验通过后，系统调用智能合约，将 ECID、订单编号、批次编号、设备类型、测试报告哈希及状态等信息写入区块链，并将生成的交易哈希回填至设备记录中。

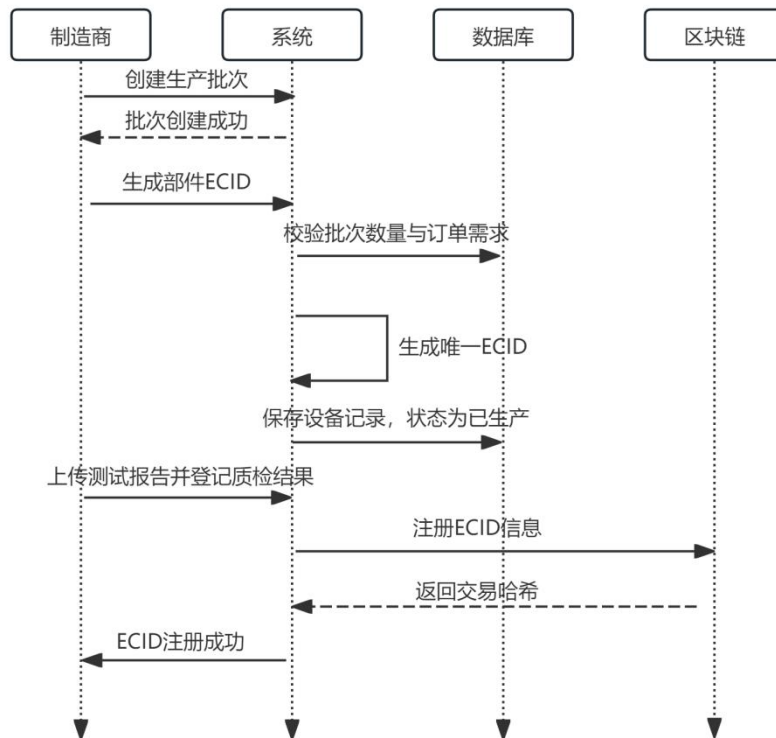


图 3-3 生产测试与 ECID 注册时序图

3.4.3 部件到整机组装映射设计

组装商依据生产订单或既定组装计划创建组装批次，并从可用的 ECID 池中选取待组装的部件。组装前，系统会逐一校验所选 ECID，确保其已完成生产质检并完成链上登记，同时未被其他整机重复占用。

在生成整机记录时，系统要求组装商填写或自动生成整机 SN，上传整机质检报告，并录入对应的固件或系统版本信息。每个整机 SN 对应一组 ECID 列表，用于描述部件与整机之间的一对多关联关系。

在组装记录提交前，系统会将整机 SN、ECID 列表、组装批次编号、固件版本以及整机质检报告哈希等信息一并提交至智能合约，完成 ECID 与 SN 的绑定操作。组装完成后，整机状态更新为“已上链”，相关部件状态同步变更为“已组装”，组装批次的完成数量随之增加；当实际完成数量达到计划值时，批次状态将更新为“已完成”。

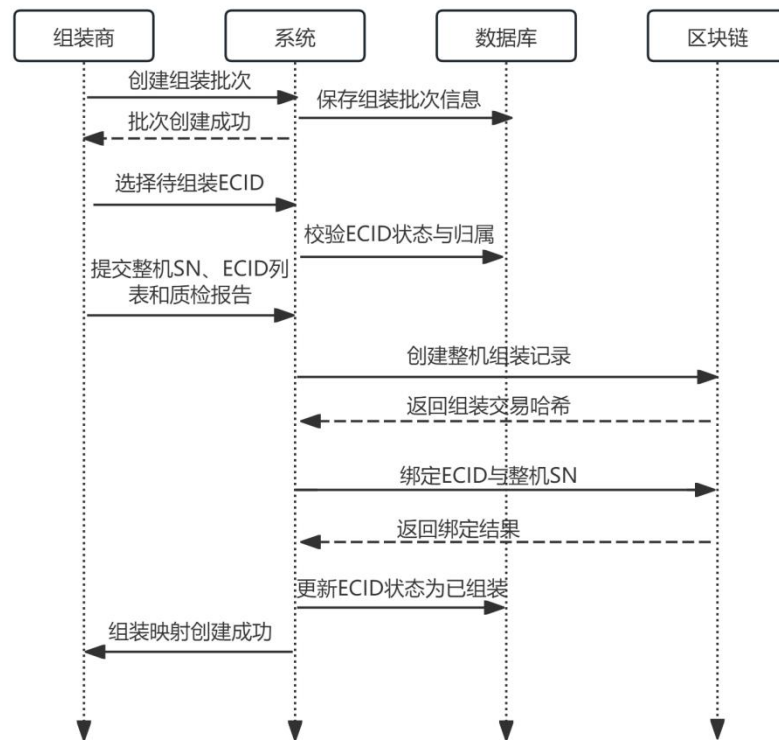


图 3-4 部件到整机组装映射时序图

3.4.4 分销流通与物流状态变更设计

系统以整机 SN 作为流通的基本对象，通过物流事件对每一次发货与收货进行记录。在发货前，系统会校验整机记录是否存在、当前操作用户是否为产品的实际持有人，以及产品状态是否允许流转；对于已销售或已报废的设备，则禁止再次发货。

发货时，发送方需填写接收方信息、物流公司、运单号、发货时间及预

计到达时间等内容。系统据此生成物流流转记录，将状态标记为“在途”，并调用智能合约完成发货事件的链上登记，同时同步更新整机状态为“在途”。此外，系统限定同一 SN 在同一时间仅能对应一条未完成的物流记录，以避免重复发货或货权状态不一致的问题。

在收货确认阶段，收货方可通过物流记录编号、运单号或 SN 查询待签收信息。系统会核对该记录是否归属于当前用户，随后将物流状态更新为“已收货”，记录实际到达时间，并生成收货摘要上链存证。完成收货后，整机的当前持有人更新为收货方，设备状态相应调整为“在库”。

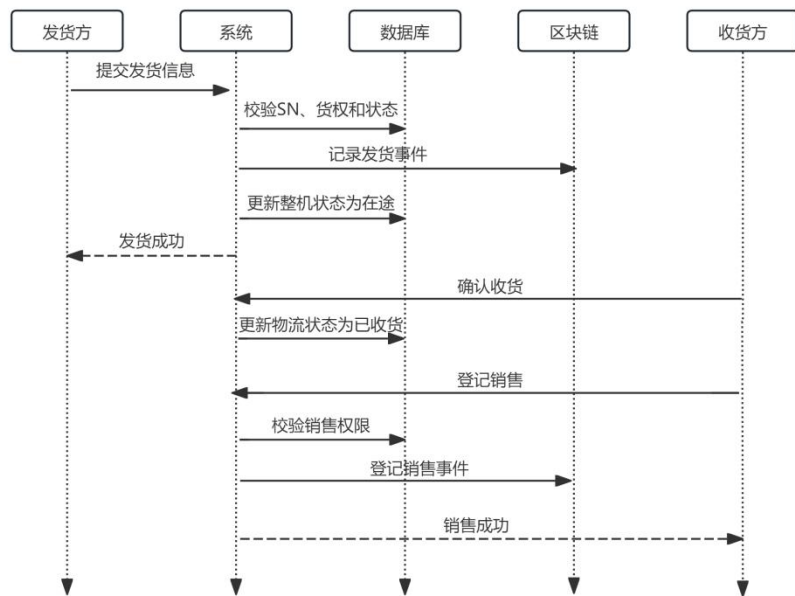


图 3-5 分销流通与物流状态时序图

3.4.5 售后与生命周期闭环设计

终端用户在完成产品购买后，可通过整机 SN 进行绑定操作。系统首先判断该 SN 是否已存在对应的销售记录，并依据购买时登记的姓名与手机号计算身份哈希，与销售记录中的客户哈希进行匹配。校验通过后，生成用户与产品的绑定关系，并将该绑定行为上链留存。用户或监管方均可基于

整机 SN 查询产品的溯源信息，系统会整合组装记录、ECID 列表、物流流转数据、销售记录以及部件级生产与质检信息，构建完整的追溯视图。

当用户发现产品存在故障时，可提交投诉或发起召回申请，填写故障类型、具体描述并上传相关证据。系统为该请求分配唯一编号，在完成证据文件存储后记录其哈希与存储地址，并将投诉内容同步上链存证。监管机构或责任方可依据问题 SN、相关 ECID 或批次信息发布召回公告，系统结合 ECID 与 SN 之间的映射关系计算受影响的整机范围，并将对应设备状态标记为“召回中”。

在产品生命周期的末端阶段，需要进行报废登记。用户或回收机构提交报废申请时，系统会校验整机档案是否存在以及是否已被重复报废。校验通过后，记录处置方式、回收主体及处理时间等信息，并通过智能合约完成报废事件的链上登记。报废完成后，整机状态更新为“已报废”，系统将禁止其继续参与流通或销售。

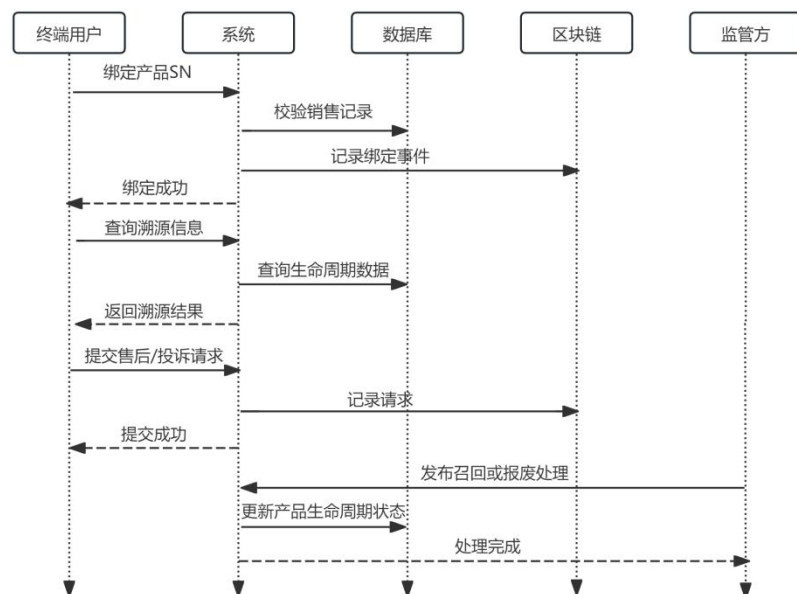


图 3-6 售后与生命周期闭环时序图

4 全流程可追溯供应链系统实现

4.1 开发环境部署与技术栈选型

4.1.1 区块链底层网络搭建

（1）FISCO BCOS 节点搭建：开发测试阶段采用单机四节点组网模式，模拟联盟链中监管机构、制造商联盟、质检机构等多方共同维护账本的部署形态。FISCO BCOS 部署在 CentOS7 环境下，配置 Java 8 运行环境以及 OpenSSL、curl 等基础工具。同时搭配可视化的 Web 管理平台 WeBASE。在项目运行时启动命令如图 4-1 所示。启动成功后可通过 `ps -ef | grep fisco` 命令查看节点进程，通过控制台连接验证网络状态。

```
//启动FISCO BCOS 节点
[root@localhost fisco]# bash nodes/127.0.0.1/start_all.sh
try to start node0
try to start node1
try to start node2
try to start node3
node2 start successfully
node0 start successfully
node1 start successfully
node3 start successfully
//启动WeBASE 平台
[root@localhost dist]# bash start.sh
=====
Server com.webank.webase.front.Application Port 5002 ...PID(3026) [Starting].
=====
```

图 4-1 区块链网络启动

智能合约文件 `SupplyChainTraceability.sol` 通过 WeBASE 编译部署，并将编译产物接口说明书（Application Binary Interface, ABI）、EVM 字节码、合约地址保存。

（2）节点与 SDK 连接：应用侧通过官方 Java SDK 读取 `config.toml`，配置证书目录、节点 `peers` 列表及发交易账户，如图 4-2 所示。合约编译

产物放入 `conf/abi`、`conf/bin`，链上部署得到的地址写入 `contract-address`。

联调阶段可通过调试接口发起只读调用或测试 `anchor`，确认证书、端口与 `Group` 配置无误。

```
[network]
# ===== 修改为你的 CentOS 7 虚拟机 IP =====
peers=["192.168.44.135:20200", "192.168.44.135:20201"]

[account]
keyStoreDir = "conf/account"           # 账户文件目录
accountFileFormat = "pem"              # 账户格式 pem / p12
accountAddress = "0x2995ae2ee2e634f471e67e4a477fe6b9b1bb0e51"# 指定发交易账户地址

[threadPool]
maxBlockingQueueSize = "102400"
```

图 4-2 config.toml 文件

4.1.2 后端应用开发技术栈

系统后端采用 Spring Boot 2.7.14 构建，配合 Spring Security + JWT 实现无状态认证授权。持久化层基于 MyBatis-Plus 与 MySQL，集成 FISCO BCOS 区块链网络，并使用 IPFS 离链存储大文件。

表 4-1 后端主要技术栈

层次	技术	在本系统中的作用
框架	Spring Boot 2.7.14	后端开发框架
安全	Spring Security + JWT	登录态、角色接口隔离
持久化	MyBatis-Plus、MySQL	订单、装配、物流、销售等业务表
链客户端	FISCO BCOS Java SDK	合约加载、发送交易、交易回执
离链存储	IPFS	设计图、质检报告、资质报告

项目地址：<https://github.com/kilingyou/GraduationProject.git>

4.2 核心智能合约的编写与部署

本系统的智能合约文件 SupplyChainTraceability.sol 由 Solidity 语言编写。设计上将角色管理、生产订单与制造协议、器件（ECID）与质检、整机组装与映射、分销物流与货权、销售摘要、召回与检验、报废与生命周期闭环等功能收敛在单一合约中。

4.2.1 角色权限与资质审核合约实现

合约定义六类角色常量：供应商、制造商、组装商、分销商、监管机构、终端用户。使用 `mapping (address => uint8) roleOf` 将链上地址与角色绑定，业务函数通过 `onlyRole`、`onlyAssemblerOrDistributor` 等修饰符约束调用方，只有合法身份才能调用相应函数。合约还提供通用锚定 `anchor(bizType, payloadHash)`，为任意“业务类型”与“内容摘要”建立序号递增的链上记录，支持事后按序号 `getAnchor` 查询操作者与时间戳。

监管方通过 `approveSupplier(supplier, qualHash)` 将供应商地址标记为已准入，并写入资质材料哈希 `_supplierQualHash`。

图 4-3 供应商资质审核函数

```
// 监管商通过后，供应商在链上获得合法地址
function approveSupplier(address supplier, string qualHash) public onlyRole(ROLE_REGULATOR) {
    approvedSuppliers[supplier] = true;
    _supplierQualHash[supplier] = qualHash;
    emit SupplierApproved( supplier: supplier,    qualHash: qualHash,    approver: msg.sender,    ts: now);
}
```

4.2.2 订单流转与生命周期管理合约实现

（1）生产需求与制造协议：供应商调用 `createProductionRequest` 创建以 `orderId` 为主键的生产需求，上链字段包括目标制造商地址、BOM 与设计文档哈希、数量、交期、质量要求哈希等，初始状态为“CREATED”，发出

ProductionRequestCreated。制造商通过 signManufacturingAgreement 写入协议哈希、价格条款与交期，置状态为“AGREED”。

（2）生产与批次闭环：制造商通过 registerDeviceRecord 按 ECID 登记单台设备及其订单、批次、类型、质检报告哈希与状态；updateDeviceStatus、recordProductionComplete、recordReject 分别支撑过程状态更新、批次结论及不合格处置记录。不合格时 _batchRejected[batchId] 置位，便于后续流转与召回策略关联。

图 4-4 注册设备函数

```
function registerDeviceRecord(  
  string ecid,  
  string orderId,  
  string batchId,  
  string devType,  
  string testReportHash,  
  string status  
) public onlyRole(ROLE_MANUFACTURER) {  
  require(bytes(ecid).length > 0, "empty ecid");  
  require(!_devices[ecid].exists, "ecid exists");  
  _devices[ecid] = DeviceRecord(  
    exists: true,  
    orderId: orderId,  
    batchId: batchId,  
    manufactureTimestamp: now,  
    manufacturer: msg.sender,  
    deviceType: devType,  
    testReportHash: testReportHash,  
    status: status  
  );  
  emit DeviceRegistered( ecid: ecid, orderId: orderId, batchId: batchId, devType: devType,  
    testReportHash: testReportHash, mfg: msg.sender, ts: now);  
}
```

（3）组装、流通与销售：组装商 createAssemblyRecord 将整机序列号 SN 与部件 ECID 列表及批次、固件版本、报告哈希一并上链；_bindEcidToSn 建立部件到整机的映射。分销商或组装商 logTransfer 记录物流单号、接收方与流转类型，并维护 SN 的当前权属地址 _snToCurrentOwner。分销商 registerSale 将客户与发票相关摘要上链，完

成 从生产到终端销售 的链上事件串联。

图 4-5 组装记录函数

```
function createAssemblyRecord(  
  string sn,  
  string ecidListJson,  
  string batchNo,  
  string fwVersion,  
  string reportHash  
) public onlyRole(ROLE_ASSEMBLER) {  
  _assemblies[sn] = AssemblyRecord( exists: true, sn: sn, ecidListJson: ecidListJson, batchNo: batchNo,  
    fwVersion: fwVersion, reportHash: reportHash, assembler: msg.sender, ts: now);  
  _snToEcidListJson[sn] = ecidListJson;  
  emit AssemblyRecordCreated( sn: sn, batchNo: batchNo, ecidListJson: ecidListJson, assembler: msg.sender, ts: now);  
}
```

4.2.3 溯源与召回机制合约逻辑实现

合约通过 `_ecidToSn`、`_snToEcidListJson`、`_devices`、`_assemblies` 等映射结构维护关键索引关系。其中，`_ecidToSn` 用于标识单个部件 ECID 所归属的整机 SN，`_snToEcidListJson` 保存整机对应的部件列表信息，`_devices` 则记录部件相关的生产订单、批次、制造商、测试报告哈希以及状态等数据。借助这些链上索引，可将原本分布在生产、组装与销售等不同环节的数据进行关联，构建可验证的追溯链路。

在溯源查询方面，合约提供 `getSnEcidList`、`getEcidSn`、`getAssemblyInfo`、`getDeviceInfoV2`、`getSnOwner`、`getSaleInfo` 等只读方法。终端用户或监管机构输入整机 SN 后，系统首先获取对应的组装信息，得到其关联的 ECID 列表；随后基于各 ECID 查询生产批次、制造商地址、设备类型以及质检报告哈希等内容；再结合物流流转记录与销售摘要，最终形成完整的产品生命周期视图。

在召回请求阶段，当用户发现产品异常时，可提交 SN、故障类型及详细描述。后端在保存投诉记录及证据文件哈希后，会调用合约中的 `requestRecall` 方法触发 `RecallRequested` 事件。该事件包含问题产品 SN、

故障信息、请求发起者地址及时间戳等内容，为后续监管审核提供不可篡改的依据。若投诉材料涉及图片或检测报告等大文件，则原始内容存储于 IPFS，链上仅保留摘要或事件记录，以平衡存证可信性与存储开销。

监管机构在确认缺陷范围后，可调用 `publishRecallNotice` 方法发布召回通告。该操作要求调用者具备监管角色权限 `onlyRole(ROLE_REGULATOR)`，以避免非授权主体伪造召回信息。通告内容包括编号及受影响的整机 SN 列表，并通过触发 `RecallNoticePublished` 事件将相关信息记录在链上。具体实现中，受影响 SN 的计算由后端完成：若已知问题 ECID，则可直接定位其所属整机；若以问题批次为依据，则需先获取该批次下全部 ECID，再结合组装记录中的 ECID 列表进行匹配，筛选出包含缺陷部件的整机 SN，并最终写入召回通告。

4.3 用户前端与核心界面开发实现

4.3.1 注册模块

供应商注册页 `register.vue` 支持营业执照与资质证书分槽上传，表单提交，如图 4-6 所示。其它不同角色注册与之类似，仅少了营业执照与资质证书的上传。对于供应商注册功能，后端计算文件哈希，将相关资质文件存入 IPFS 并返回 CID，此时供应商状态为待审核（PENDING），供应商注册阶段不立即写入链上角色，链上角色映射在监管机构审核通过后完成。供应商注册登录后，订单页在调用发布接口前需校验“资质已通过”。



企业注册
注册成为供应链平台成员

* 登录账号
请输入账号

* 角色
供应商

* 密码
请输入密码

* 确认密码
请再次输入密码

企业名称
请输入企业名称

统一社会信用代码
请输入信用代码

联系人
请输入联系人

联系电话
请输入联系电话

电子邮箱
请输入电子邮箱

营业执照
上传营业执照

资质证书（可选）
上传资质证书

支持 PDF/JPG/JPEG/PNG。营业执照与资质证书分开上传，提交时会按顺序传给后端。

提交注册

已有账号？[返回登录](#)

图 4-6 供应商注册界面

4.3.2 供应商模块

供应商模块核心功能包括：上传设计文档、发布 BOM 清单、声明安全合规、发起生产订单。所有功能首先需要通过资质审核校验才能进行。

实现中，供应商上传设计文档，先存 IPFS，设计文档文件哈希与版本号通过通用 anchor 方法写入区块链。发布 BOM 清单时，由前端填入的信息以及设计文档组装上链清单 manifest 对象，并将 manifest 对象序列化为 JSON 字节数组，最终调用 `evidenceStorageService.store` 方法实现上链。发布订单时如图 4-7 所示，后端创建生产订单，并设置订单状态为待接单（`PENDING_ACCEPTANCE`），调用智能合约的 `createProductionRequest` 方法，传入 BOM 哈希、设计哈希、数量、交期等参数，合约生成

ProductionRequest 结构体并触发 ProductionRequestCreated 事件。

发布生产订单

* 选择BOM

请选择BOM

* 生产数量

-

1

+

* 期望交期

选择日期

质量要求

请输入质量要求

目标制造商

不选则对所有制造商可见 (广播)

取消

确认发布

图 4-7 供应商发布订单

4.3.3 制造商模块

制造商模块负责接收供应商订单、签署生产协议、生成物理唯一标识（ECID）、质检合格后上链登记。制造商登录后可见分配给自己的订单，点击“接单”，填写最终报价、交付日期，上传协议文件。后端更新订单状态为 ACCEPTED，通过 anchor 上链协议文件。

订单接收

制造商 mfg_test05

订单大厅 (待接单)

我的订单 (已接单)

搜索订单ID / BOM名称

订单ID	BOM	设计文档	设计文档哈希	图纸	数量	操作
129524977909...	移动电话物...	移动电话设...	0681e51f16181de...	-	1	接单

共 1 条

10条/页

< 1 >

前往 1 页

图 4-8 制造商接单

生产环节中，制造商按批次批量生成 ECID，如图 4-9 所示。每个 ECID 对应一个 DeviceRecord 结构体，包含订单 ID、批次号、生产时间、交易哈

希、状态（合格/不合格）等。质检通过则上链注册，若不合格，调用 recordReject 触发退货流程。

批次管理 ECID管理					
* 批次	选择批次 (新批次/选择历史批次)	* 数量	10	* 设备类型	选 Sensor A
批量生成 ECID					
ECID	订单	批次	设备类型	状态	操作
☐ ECID-M0024-20260412-000025	3-0808421aac4439b70ca3738a701b	BATCH-20260412-5185345A	PCB板	质检通过	上链
☐ ECID-M0024-20260412-000030	3-0808421aac4439b70ca3738a701b	BATCH-20260412-5185345A	PCB板	质检通过	上链
☐ ECID-M0024-20260412-000028	3-0808421aac4439b70ca3738a701b	BATCH-20260412-5185345A	PCB板	质检通过	上链
☐ ECID-M0024-20260412-000026	3-0808421aac4439b70ca3738a701b	BATCH-20260412-5185345A	PCB板	质检通过	上链
☐ ECID-M0024-20260412-000029	3-0808421aac4439b70ca3738a701b	BATCH-20260412-5185345A	PCB板	质检通过	上链
☐ ECID-M0024-20260412-000024	3-0808421aac4439b70ca3738a701b	BATCH-20260412-5185345A	PCB板	质检通过	上链
☐ ECID-M0024-20260412-000027	3-0808421aac4439b70ca3738a701b	BATCH-20260412-5185345A	PCB板	质检通过	上链
☐ ECID-M0024-20260412-000023	3-0808421aac4439b70ca3738a701b	BATCH-20260412-5185345A	PCB板	质检通过	上链
☐ ECID-M0024-20260412-000022	3-0808421aac4439b70ca3738a701b	BATCH-20260412-5185345A	PCB板	质检通过	上链
☐ ECID-M0024-20260412-000021	3-0808421aac4439b70ca3738a701b	BATCH-20260412-5185345A	PCB板	质检通过	上链

图 4-9 制造商批量生成 ECID

4.3.4 组装商模块

组装商模块的核心是将多个 ECID（电子元器件或 PCB 板）组装为一个整机，生成新的产品序列号（SN），并记录固件版本、整机测试报告，此外组装商还支持通过 ECID 检测器件是否质检通过，如 4-10 所示。

验证通过

ECID: ECID-M0030-20260503-000003 — 该器件已通过质检，可用于组装

器件类型	1 摄像头	生产批次	BATCH-20260503-FB4D4F61
生产订单	1295249779094d76acc7d65fbad9d71a	BOM 子件	1 / 摄像头
说明	验证通过，可用于组装		
部件链上	已注册		

图 4-10 验证器件合法性

组装商填入生产订单号、产品型号、计划生产数量创建组装批次；其次选择整机所需的 ECID 列表创建组装记录（AssemblyRecord）并生成整机

SN 号，质检通过后注册上链。

组装批次

组装记录

批次筛选

全部批次

导出 CSV

一键导出 SN

* 组装批次

ASM-20260413-19FB774A (移动电话...

* 固件版本

v1.0

整机 SN

选填，不填则系统自动生成

* ECID列表

ECID-M0030-20260413-000013 (3 电池 · BATCH-20260413-39957E65) ×

ECID-M0030-20260413-000010 (2 内存 · BATCH-20260413-2A099FFC) ×

ECID-M0030-20260413-000003 (1 摄像头 · BATCH-20260413-14A9591F) ×

创建记录

SN	批次号	ECID列表	固件版本	操作
SN-20260503-275171	ASM-20260503-45ACD09D	ECID-M0030-20260503-000004 ECID-M0030-20260503-000003	v1.0	已上链
SN-20260421-589417	ASM-20260417-3AF42EEB	ECID-M0030-20260416-000003 ECID-M0030-20260416-000002 ECID-M0030-20260416-000001	v1.0	注册上链
SN-20260413-966875	ASM-20260413-19FB774A	ECID-M0030-20260413-000012 ECID-M0030-20260413-000009 ECID-M0030-20260413-000002	v1.0	已上链
SN-20260413-266382	ASM-20260413-19FB774A	ECID-M0030-20260413-000011 ECID-M0030-20260413-000008 ECID-M0030-20260413-000001	v1.0	已上链

共 4 条

10条/页

< 1 >

图 4-11 组装商创建组装记录

4.3.5 分销商模块

分销商模块位于供应链系统中游，负责承接上游组装出库、完成渠道流转、登记终端销的核心职责。该模块以产品唯一序列号（SN）为业务主键，围绕库存、物流、销售三类场景实现业务，并通过链上锚定机制保障关键事件的可追溯性和不可抵赖性。

分销商首先对组装完成的整件进行收货，将物流记录状态改为已签收（RECIIVED），生成签收摘要并链上锚定。然后进行销售登记，目标客户分为企业客户与终端客户，同时为了保护客户隐私，销售登记时可以选择

是否匿名销售，匿名模式仅存用户摘要信息。最后上传发票附件创建销售记录上链。

登记销售

×

* 产品SN

请输入产品 SN

* 销售时间

🕒 选择销售时间

客户类型

企业客户 (B2B)

终端用户 (B2C)

匿名销售

否

☐

是

开启后不保存客户姓名/电话明文，仅链上摘要

客户姓名

可选，脱敏存储

客户电话

可选，脱敏存储

发票附件

选择文件

取消

确认登记

图 4-11 登记销售

4.3.6 终端用户模块

终端用户模块面向产品使用者，核心功能包含溯源查询、投诉反馈、产品绑定、报废登记功能，其在系统中承担“消费端可信交互层”。一方面确认用户与产品的合法关系，另一方面将投诉、报废等后市场事件纳入可追踪链路。对于溯源查询，按 SN 聚合多维数据，包含组装记录、物流流转、销售记录,统一封装到 map 集合中并返回到前端显示。此外支持 ipfsCid 和 expectedHash 做文件内容校验，实现全流程可追溯验证。

输入产品序列号(SN)查询真伪与溯源

SN: 20200421-589417

无图验证码可省略: 请关闭浏览器窗口或刷新页面, 防止恶意篡改数据。

返回验证 记录历史 扫一扫: QR Code

⚠️ 请确认设备序列号与平台数据库一致, 避免数据不一致。

输入溯源: 输入ECID或SN进行溯源

输入 ECID: ECID-400030-20200415-000003

输入 ECID: ECID-400030-20200415-000002

输入 ECID: ECID-400030-20200415-000001

溯源信息:

整机 SN	SN-20200421-589417	整机批次	ASMA-20200417-5AF-02E8B
固件版本	v1.0	固件哈希值	0d91d11f161816d1f6c0a027b6d05774d277b6d0577b0337b6d0581203
链上状态	SOLD	链上哈希	0x1767b07439f13c0e9d2a795e7a8b123338f167d7275d053e5410d6

一键生成事件链哈希值 (SHA256 + HECV + ECID) 从非链上数据生成事件链哈希, 与链上记录对比 (与图章一致为一致)。

操作 ECID 列表

ECID-400030-20200415-000003 ECID-400030-20200415-000002 ECID-400030-20200415-000001

溯源记录

类型	溯源记录	溯源事件	溯源时间	溯源地点	溯源设备	溯源时间
SN	整机溯源	1589+	2020-04-21 17:00:00	20	32	2020-04-21 17:00:00

溯源信息:

溯源时间	2020-04-21 17:00:00	客户类型	SCC
溯源设备	否	客户地址	7ba3d8a4e93e02a71e697f7b6d0581203a02b0e12b6d2c0e0
溯源哈希	0d91d11f161816d1f6c0a027b6d05774d277b6d0577b0337b6d0581203	溯源 ID	0x1767b07439f13c0e9d2a795e7a8b123338f167d7275d053e5410d6
溯源链上 Tx	0x1767b07439f13c0e9d2a795e7a8b123338f167d7275d053e5410d6		

校验数据 重置数据

图 4-12 溯源查询

对于产品绑定, 用户前端提交 SN、姓名、电话, 系统将用户输入的信息计算哈希与销售记录中的 `customerHash` 比对, 一致则将状态设为已验证 (VERIFIED), 并生成绑定事件链上哈希。

4.3.7 监管机构模块

监管机构模块具备最高权限, 主要功能包括: 供应商资质审核、抽检任务下发、召回管理、审计日志查看。审核时, 监管人员查看供应商上传的 IPFS 文件, 决定“通过”或“拒绝”, 结果更新本地数据库并上链存证。抽检功能中, 监管机构可指定某 ECID、批次号进行重新检测, 对检测报告哈希上链。召回功能根据 `faultEcid` 或 `faultBatchID` 解析目标缺陷部件集合, 反向定位受影响的整机 SN, 将其状态标记为召回 (RECALLING), 生成召回通告并链上摘要。

5 系统测试与分析

5.1 核心功能测试

系统功能测试部分主要对供应商资质审核、订单流转、生产登记、组装映射、溯源查询等进行测试。

（1）供应商资质审核测试：供应商前端填写信息并上传资质文件至 IPFS，监管机构登录，查看 IPFS 文件并点击“审核通过”。测试用例如表 5-1 所示。

表 5-1 供应商资质审核测试用例

测试前提	监管机构账号已注册
测试步骤	1) 填写信息，并上传资质文件
	2) 监管机构点击审核通过
预期结果	1) 文件成功存入 IPFS,生成 CID
	2)智能合约记录供应商地址及资质哈希，状态变更为“已准入”
测试结果	符合预期，链上成功生成 SupplierApproved 事件，凭证可查。

通过本地数据库中存储的交易哈希使用浏览器访问 <http://localhost:5173/api/debug/fisco/tx/<txHash>>得到的具体内容如图 5-1 所示。

```

{
  "code": 200,
  "message": "success",
  "data": {
    "time": "2026-05-03T15:11:57.765",
    "txHash": "0xf73f4280570e4bb2bcd5e045f99eee4e7bcfcbeec2adcab9af833f59b6bf3c6",
    "blockchainMode": "fisco",
    "found": true,
    "status": "0x0",
    "statusOk": true,
    "statusMsg": "None",
    "blockNumber": "0x556",
    "blockHash": "0x6c6b43dbbc5a3282c4b0e3cc178b60747292767489aaaaac13fd9342e544be128",
    "from": "0x3c005025c013de6f13c3d7d49f6ce5a2774fb8e4",
    "to": "0x7e5b0ec00ccb1c94248f419570296cba6898db5b",
    "gasUsed": "0x1b7c4",
    "contractAddress": "0x0000000000000000000000000000000000000000000000000000000000000000",
    "logsCount": 1,
    "ok": true,
    "message": "Transaction receipt query success"
  }
}

```

图 5-1 资质审核交易查询

(2) 订单流转测试：验证订单创建与制造协议签订，供应商完成设计文档上传、BOM 清单创建后，输入数量、交期、目标制造商等信息发布订单；制造商查收订单，上传协议文件并确认接单。

表 5-2 订单流转测试用例

测试前提	供应商资质审核通过
测试步骤	1) 上传设计文档，创建 BOM 清单
	2) 发布生产清单
	3) 制造商确认接单
预期结果	1) 订单状态依次变更为 CREATED、AGREED
	2) 协议文件哈希及双方签名上链，不可抵赖
测试结果	符合预期，调用 createProductionRequest 及接单合约成功

与前文相同，访问制造协议交易哈希得到的具体内容如图 5-2 所示。


```
{
  "code": 200,
  "message": "success",
  "data": {
    "time": "2026-05-03T16:10:39.719",
    "txHash": "0x607aa7409b11b683e71c3ede793264e1eab8bb3bc6216962770f24181297042d",
    "blockchainMode": "fisco",
    "found": true,
    "status": "0x0",
    "statusOk": true,
    "statusMsg": "None",
    "blockNumber": "0x55c",
    "blockHash": "0xb31067ba54f387ad213d0327c13c4bc5e2ff4faab72470c859cd895b249de45d",
    "from": "0xae398d3989aa0f39967853b5747aec7d571f04c5",
    "to": "0x7e5b0ec00ccb1c94248f419570296cba6898db5b",
    "gasUsed": "0x46d47",
    "contractAddress": "0x0000000000000000000000000000000000000000000000000000000000000000",
    "logsCount": 1,
    "ok": true,
    "message": "Transaction receipt query success"
  }
}
```

图 5-2 制造协议交易查询

（3）生产登记测试：验证 ECID 生成与设备上链，制造商按批次生成 ECID，上传质检报告后调用上链注册接口。

表 5-3 生产登记测试用例

测试前提	制造商已接单
测试步骤	1) 制造商根据订单创建生产批次
	2) 制造商根据生产批次生成 ECID
	3) 上传质检报告
	4) 将设备注册上链
预期结果	链上成功创建 DeviceRecord，保存订单 ID、批次、时间戳和状态等，重复注册报错
测试结果	符合预期，设备身份在链上唯一确立

与前文相同，访问设备记录哈希得到的具体内容如图 5-3 所示。

```
{
  "code": 200,
  "message": "success",
  "data": {
    "time": "2026-05-03T16:52:58.199",
    "txHash": "0xa9c5401298bf2fb65a499d24577826241cf74671ebb5965671b6601611177c1c",
    "blockchainMode": "fisco",
    "found": true,
    "status": "0x0",
    "statusOk": true,
    "statusMsg": "None",
    "blockNumber": "0x56f",
    "blockHash": "0x6956f461dc8f790e15f7ecdfb81c81a1f32f4a4bf902e4363209d7609a0eb833",
    "from": "0xae398d3989aa0f39967853b5747aec7d571f04c5",
    "to": "0x7e5b0ec00ccb1c94248f419570296cba6898db5b",
    "gasUsed": "0x41b18",
    "contractAddress": "0x0000000000000000000000000000000000000000000000000000000000000000",
    "logsCount": 1,
    "ok": true,
    "message": "Transaction receipt query success"
  }
}
```

图 5-3 设备记录交易查询

（4）组装映射测试：验证部件到整机的绑定逻辑，组装商输入整机 SN，勾选对应的部件 ECID 列表，提交组装记录。

表 5-4 组装映射测试用例

测试前提	组装所需设备的 ECID 列表已经质检通过，注册上链
测试步骤	1）创建组装批次
	2）创建组装记录
预期结果	系统成功建立“部件 ECID->整机 SN”的映射，链上写入 AssemblyRecord。
测试结果	符合预期，支持通过 SN 反查所有组成部件

与前文相同，访问组装记录哈希得到的具体内容如图 5-4 所示。

```
{
  "code": 200,
  "message": "success",
  "data": {
    "time": "2026-05-03T18:12:52.805",
    "txHash": "0x4c8cb30c7a14b148fb28a94ef2d90a21670af12b90c13eb493bee34f6b4d4eb7",
    "blockchainMode": "fisco",
    "found": true,
    "status": "0x0",
    "statusOk": true,
    "statusMsg": "None",
    "blockNumber": "0x5a9",
    "blockHash": "0x8c97c7894f6895cb0ae3fc7965797c68e28f9d824f777cde332f90d98c4e7028",
    "from": "0x27910371f4905544dcbf9f4e1cabfad964e84ea0",
    "to": "0xb628b9140bbd1d4ecccc991434db7025d18a7836",
    "gasUsed": "0x5f7e4",
    "contractAddress": "0x000000000000000000000000000000000000000000000000",
    "logsCount": 1,
    "ok": true,
    "message": "Transaction receipt query success"
  }
}
```

图 5-4 组装记录交易查询

（5）溯源查询测试：验证终端用户防伪与溯源展示，终端用户在查询页面输入产品 SN 或扫描产品二维码。

表 5-5 溯源测试用例

测试前提	已整机组装
测试步骤	前端输入产品 SN
预期结果	前端时间线可视化展示该 SN 的设计、生产、组装、物流及销售全过程，并支持校验文件哈希。
测试结果	符合预期，若文件被篡改，前端提示“哈希校验失败”；无篡改则显示验证通过。

终端用户通过 SN 查询得到的查询结果如图 5-5 所示。

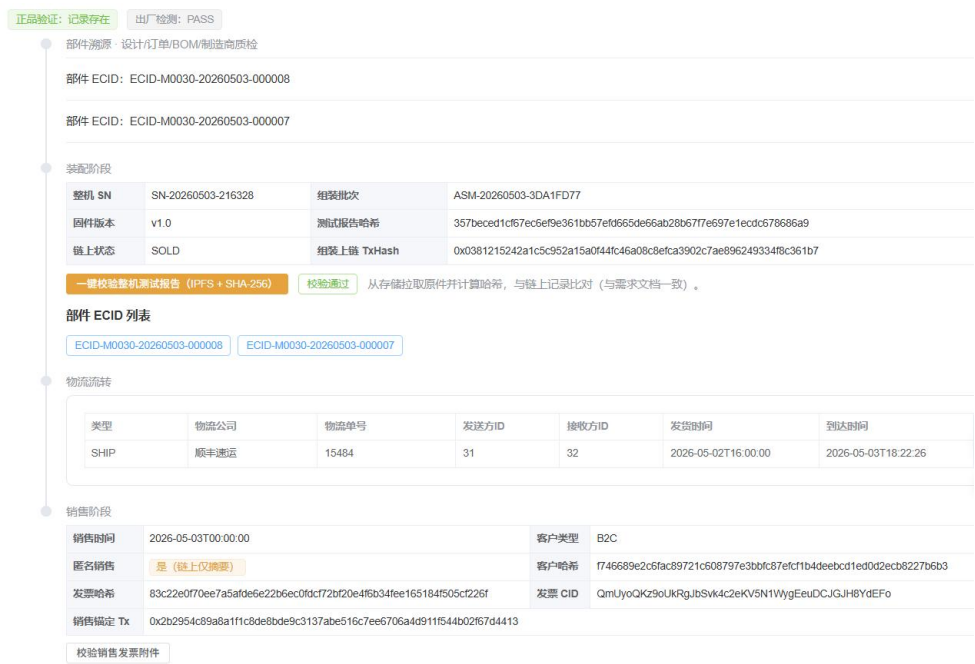


图 5-5 溯源查询结果

5.2 性能测试

为验证系统在多角色协同场景下的稳定与响应能力，本节对供应链溯源系统进行性能测试。重点关注普通业务查询、业务写入以及区块链存证类接口在并发访问下的响应情况。

5.2.1 测试环境

测试在本地局域网环境进行，测试前数据库预置供应商、制造商、组装商、分销商、监管机构和终端用户等角色数据，并构造订单、生产、组装、销售等业务数据，模拟系统实际运行时的数据规模，测试环境配置如表 5-6 所示。

表 5-6 性能测试环境

项目	配置
操作系统	Windows11
后端环境	JDK 1.8, Spring Boot 2.7.14
数据库	MySQL 8.0, IPFS
前端环境	Vue3, Vite, Element Plus
区块链环境	FISCO BCOS 2.11.0
测试工具	Apache JMeter
浏览器	Chrome
测试网络	本地局域网

5.2.2 测试指标

本次性能测试主要关注以下指标，如表 5-7 所示。其中，普通查询类接口要求响应时间较短；写入类接口响应时间允许略高；区块链存证接口需要等待交易完成，耗时通常高于普通数据库操作。

表 5-7 测试指标

指标	说明
平均响应时间	接口请求从发起到返回结果的平均耗时
95%响应时间	95%请求能够完成响应的最大耗时
吞吐量	单位时间内系统能够处理的请求数量
错误率	请求失败数量占总请求数量的比例

5.2.3 测试结果

本次测试采用阶梯式并发方式，分别设置 10、30、50 和 100 个并发用户，每组测试持续 5 分钟。主要的测试接口为用户登录、数据看板、生产批次查询、组装记录查询、产品溯源查询、区块链存证等接口。其中并发用户数量为 50 时，接口性能测试结果如表 5-8 所示。

测试场景	平均响应时间 /ms	95%响应时间 /ms	吞吐量/次每秒	错误率
用户登录	137	155	5.03525	0.00%
数据看板	8	13	5.09788	0.00%
生产批次查询	6	8	5.09632	0.00%
组装记录查询	6	9	5.09476	0.00%
产品溯源查询	17	21	5.09684	0.00%
区块链取证	411	653	4.84168	0.00%

表 5-8 接口性能测试结果

从测试结果可以看出，在 50 并发用户场景下，系统普通查询类接口平均响应时间基本保持在 20ms 以内，能够满足日常管理操作需求。写入类接口保持在 200ms 以内，区块链存证接口由于需要与 FISCO BCOS 节点交互，平均响应时间明显高于普通数据库接口，但错误率为 0%，说明系统在测试压力下能够稳定完成链上存证操作。

6 总结与展望

6.1 工作总结

本文围绕电子产品供应链中普遍存在的数据孤岛、信息不透明及追溯困难等问题，基于区块链技术设计并实现了一套全流程可追溯的电子产品供应链管理系统。通过将区块链、智能合约与离链存储技术相结合，实现了从设计、生产、组装、流通到销售及报废的全生命周期数据可信记录与追溯。在研究与实现过程中，本文主要完成了以下几方面工作：

（1）在理论层面分析了电子产品供应链的结构及其面临的核心问题，包括信息割裂、数据易篡改及责任追溯困难等，并结合区块链去中心化、不可篡改等特性，提出了基于联盟链的供应链溯源解决思路。

（2）在系统设计方面，构建了以“链上存证+链下存储”为核心的混合架构。通过引入物理唯一标识（ECID）与产品序列号（SN），建立“部件—整机—流通”的多层级映射关系，实现了从单一元器件到整机产品的全链路追踪。同时，设计了涵盖供应商、制造商、组装商、分销商、监管机构及终端用户的多角色协同模型，并明确了各角色的功能与权限。

（3）在系统实现方面，基于 FISCO BCOS 联盟链平台，完成了智能合约的设计与部署，实现了订单管理、设备注册、组装映射、物流记录、销售登记及报废管理等核心业务逻辑。同时，结合 Spring Boot、MySQL 与 IPFS 技术，开发了完整的后端服务与前端交互界面，实现链上链下数据协同管理。

（4）在系统测试与分析阶段，对供应商资质审核、订单流转、生产登记、组装映射及溯源查询等关键功能进行了验证。测试结果表明，系统能够正确完成各业务流程，并具备良好的数据一致性与可追溯性，验证了系统设计的可行性与有效性。

综上，本文实现了一套具有实际应用价值的区块链供应链溯源系统，在提升数据可信性、增强供应链透明度以及支持精准召回方面具有一定的创新性与实践意义。

6.2 工作展望

本文已完成系统的设计与实现，并在实验环境中验证了其可行性，但仍存在一些不足之处，有待未来进一步研究与改善。

第一，在系统性能方面，当前系统基于联盟链实现，虽然相较公有链具有更高的性能，但在大规模节点与高并发交易场景下仍可能面临性能瓶颈。未来可进一步优化共识机制（如引入改进型 PBFT 或分层共识架构），并结合批处理、异步提交等策略提升系统吞吐能力^[21]。

第二，在隐私保护方面，虽然系统采用“链上摘要、链下存储”的方式避免敏感数据直接上链，但在多方协作场景下，数据访问控制仍有进一步提升空间。未来可结合零知识证明、多方安全计算等隐私计算技术，实现“可验证但不可见”的数据共享机制。例如，引入零知识证明可在不泄露原始数据的前提下完成可信验证，从而增强系统隐私保护能力^[22]。

第三，在系统安全性方面，目前系统主要依赖区块链本身的防篡改特性，对于智能合约漏洞、密钥管理等问题仍需加强。后续可引入形式化验证工

具对智能合约进行安全审计，并结合硬件安全模块或多重签名机制提升密钥管理安全性。

第四，在应用扩展方面，当前系统主要针对电子产品供应链场景，未来可将其推广至医药溯源、食品安全、工业制造等更多领域。同时，可结合物联网技术（如 RFID、传感器设备），实现生产与物流数据的自动采集，从而进一步减少人为干预，提高数据可信度。

综上所述，随着区块链技术与数字供应链的不断发展，基于区块链的供应链溯源系统将在提升产业透明度、保障产品质量及强化监管能力等方面发挥更加重要的作用，具有广阔的研究与应用前景。

致谢

谢谢。

参考文献

- [1] Ivanov D, Dolgui A. A digital supply chain twin for managing the disruption risks and resilience in the era of Industry 4.0[J]. Production Planning & Control, 2021, 32(9): 775-788.
- [2] Kshetri N. Blockchain's roles in strengthening cybersecurity and protecting privacy[J]. Telecommunications Policy, 2017, 41(10): 1027-1038.
- [3] Nakamoto S. Bitcoin: A Peer-to-Peer Electronic Cash System. (2008)[2026]. <https://ssrn.com/abstract=3440802>.
- [4] Saberi S, Kouhizadeh M, Sarkis J, et al. Blockchain technology and its relationships to sustainable supply chain management[J]. International Journal of Production Research, 2019, 57(7): 2117-2135.
- [5] Ngai E W T, Moon K K L, Riggins F J. RFID research: An academic literature review (1995 – 2005) and future research directions[J]. International Journal of Production Economics, 2008, 112(2): 510-520.
- [6] Atzori L, Iera A, Morabito G. The Internet of Things: A survey[J]. Computer Networks, 2010, 54(15): 2787-2805.
- [7] Lee J, Bagheri B, Kao H A. A Cyber-Physical Systems architecture for Industry 4.0-based manufacturing systems[J]. Manufacturing Letters, 2015, 3: 18-23.
- [8] Kshetri N. 1 Blockchain' s roles in meeting key supply chain management objectives[J]. International Journal of Information Management, 2018, 39: 80-89.
- [9] Christidis K, Devetsikiotis M. Blockchains and smart contracts for the Internet of Things[J]. IEEE Access, 2016, 4: 2292-2303.
- [10] Zheng Z, Xie S, Dai H, et al. Blockchain challenges and opportunities: A survey[J]. International Journal of Web and Grid Services, 2018, 14(4): 352-375.
- [11] 田洪亮, 宪明杰, 葛平. 基于联盟链的细粒度安全访问控制机制[J]. 计算机科学, 2024, 51(S1): 1035-1041.
- [12] Xu X L, Rahman F, Shakya B, et al. Electronics Supply Chain Integrity Enabled by Blockchain[J]. ACM Transactions on Design Automation of Electronic Systems, 2019, 24(3): 1-25.
- [13] Bhure C, Nicholas G S, Ghosh S, et al. AutoDetect: Novel Autoencoding Architecture for Counterfeit IC Detection[J]. Journal of Hardware and Systems Security, 2024, 8(3): 113-132.
- [14] Gao P F, Kong D C, Li X Q. Implementation and Security Analysis of Cryptocurrencies Based on Ethereum. (2025)[2026]. <https://doi.org/10.48550/arXiv.2504.21367>.
- [15] 陈会云, 陈建. 区块链共识算法综述[J]. 中国管理信息化, 2024, 27(10): 175-177.
- [16] 袁昊天, 李飞. 基于改进 Raft 共识算法和 PBFT 共识算法的双层共识算法[J]. 计算机应用研究, 2024, 41(05): 1314-1320.
- [17] Balasubramanian K. Security of the Secp256k1 Elliptic Curve Used in the Bitcoin Blockchain[J]. Indian Journal of Cryptography and Network Security, 2024, 4(1): 1-5.
- [18] FISCO BCOS 开源社区. FISCO BCOS 技术文档[EB/OL]. (2019)[2026]. <https://fisco-bcos-documentation.readthedocs.io/>.
- [19] Benet J. IPFS - Content Addressed Versioned P2P File System[EB/OL]. (2019)[2026]. <https://doi.org/10.48550/arXiv.1407.3561>.
- [20] Sandhu R S, Coyne E J, Feinstein H L, et al. Role-based access control models[J]. IEEE computer, 1996, 29(2): 38-47.

- [21] Wang D, Huang X, Jiang Z, et al. An Efficient Supply Chain Traceability Architecture Based on Blockchain and a Dynamic-Credit PBFT Consensus Algorithm[J/OL]. Informatica, 2025, 49(31).
<https://doi.org/10.31449/inf.v49i31.11784>
- [22] Sahai S, Singh N, Dayama P. Enabling Privacy and Traceability in Supply Chains using Blockchain and Zero Knowledge Proofs[E]. In: 2020 IEEE International Conference on Blockchain (Blockchain). Greece: IEEE, 2020. 134-143.